LAPORAN TUGAS M6
Komputasi Paralel

Implementasi dan Analisis Komparatif POSIX Threads vs OpenMP pada Simulasi Monte Carlo Estimasi Nilai π

Disusun oleh:

Adam Arias Jauhari
NIM 2322105018 | No. Absen 16

Hafidh Tazakka
NIM 2322105006 | No. Absen 06

Ahmad Daffa Nurbani Alam
NIM 2322105021 | No. Absen 18

Noval Abiludin
NIM 2322105013 | No. Absen 12

Muhammad Sayed Azis
NIM 2322105071 | No. Absen 55

Kelas 4TI01

Program Studi Teknik Informatika
Universitas Cendekia Abditama

2026

Repository: github.com/AdamJauhari/paralel-2322105018-Adam-Arias-Jauhari

1. PENDAHULUAN

1.1 Latar Belakang Masalah

Metode Monte Carlo menyelesaikan masalah numerik dengan pengambilan sampel acak berulang. Salah satu contoh klasiknya adalah estimasi nilai π: titik (x, y) diambil seragam acak pada kuadran satuan [0,1] × [0,1], lalu dihitung berapa banyak titik yang jatuh di dalam seperempat lingkaran satuan ($x^2 + y^2 \leq 1$). Karena luas seperempat lingkaran adalah π/4 dan luas kuadran adalah 1, rasio titik di dalam lingkaran terhadap total titik menghampiri π/4, sehingga $\pi \approx 4 \cdot$ (jumlah titik di dalam) / N.

Galat standar estimator ini menurun sebagai $1/\sqrt{N}$, yaitu $\approx 1{,}642/\sqrt{N}$. Untuk memperoleh satu digit desimal tambahan diperlukan sekitar 100 kali lebih banyak sampel. Konsekuensinya, estimasi yang akurat membutuhkan jumlah sampel sangat besar (pada tugas ini $N = 10^9$) dan waktu komputasi serial menjadi signifikan. Di sisi lain, setiap sampel bersifat independen sehingga masalah ini tergolong *embarrassingly parallel*: tidak ada ketergantungan data antar iterasi selain penjumlahan akhir. Karakteristik ini menjadikannya kasus yang ideal untuk membandingkan dua pendekatan pemrograman paralel pada sistem *shared-memory*.

Meskipun mudah diparalelkan, kasus ini memuat tiga jebakan yang khas dalam pemrograman paralel: (1) *race condition* pada akumulator hasil, (2) penggunaan generator bilangan acak (RNG) global yang menjadi titik kontensi sekaligus merusak kualitas statistik, dan (3) *false sharing* pada variabel hasil per-thread yang berdekatan di memori. Ketiganya dibahas pada laporan ini.

1.2 Tujuan Implementasi

- Mengimplementasikan estimasi π dengan Monte Carlo 10^9 sampel dalam tiga versi: serial (baseline), POSIX Threads (Pthreads), dan OpenMP.
- Menjamin kebenaran hasil: tidak ada *race condition*, RNG privat per thread, dan hasil paralel dapat diverifikasi terhadap versi serial.
- Mengukur waktu eksekusi, *speedup*, dan *efficiency* untuk 1, 2, 4, dan 8 thread.
- Menganalisis secara kritis perbedaan kompleksitas kode, bottleneck (Hukum Amdahl), anomali pengukuran, serta *trade-off* antara kemudahan (OpenMP) dan kontrol (Pthreads).
- Menganalisis *trade-off* jumlah thread terhadap akurasi statistik.

2. DESAIN IMPLEMENTASI

2.1 Algoritma Serial (Baseline)

Algoritma serial menggunakan satu RNG dan satu loop sepanjang N iterasi. Pseudocode-nya:

```
hits <- 0
rng  <- seed(42)
untuk i = 1 .. N:
    x <- rng_double(rng)        // seragam di [0,1)
    y <- rng_double(rng)
    jika x*x + y*y <= 1:  hits <- hits + 1
pi_hat <- 4 * hits / N
```

Kompleksitas waktu O(N) dengan memori O(1): seluruh state (4 × 64 bit RNG, *counter* hits, x, y) muat di register CPU sehingga loop tidak memproduksi lalu lintas memori yang berarti. Beban kerja ini murni *compute-bound*. Hal ini penting untuk analisis selanjutnya: kinerja paralel tidak dibatasi oleh *memory bandwidth*, melainkan oleh jumlah *core* fisik, frekuensi CPU, dan overhead paralelisasi.

Generator bilangan acak. Digunakan xoshiro256** (Blackman & Vigna, 2021): generator modern dengan state 256 bit, periode $2^{256} - 1$, lolos baterai uji statistik BigCrush, dan sangat cepat (hanya operasi geser, rotasi, XOR, dan perkalian). State diinisialisasi dari satu seed 64-bit menggunakan SplitMix64 (Steele dkk., 2014) agar bit-bit state terdistribusi baik. Bilangan *double* dibentuk dari 53 bit teratas keluaran 64-bit (dibagi 2^{53}) sehingga seragam di [0,1). Pemilihan ini menggantikan `rand()` yang memakai state global dan (pada glibc) dilindungi kunci; `rand_r()` sebenarnya thread-safe tetapi berkualitas statistik jauh lebih rendah dan *state*-nya hanya 32 bit.

2.2 Strategi Paralelisasi Pthreads

Pembagian workload

Digunakan *block partition* statis. Dengan T thread dan N sampel, thread ke-t menerima $n_t = \lfloor N/T \rfloor + [t < (N \bmod T)]$ sampel; sisa pembagian disebar satu per satu ke thread berindeks kecil sehingga selisih beban antar thread paling banyak satu sampel. Karena setiap iterasi memiliki biaya hampir identik (tidak ada percabangan berat atau data-dependent cost), partisi statis sudah menghasilkan *load balance* yang baik dan tidak memerlukan penjadwalan dinamis. Setiap thread memanggil fungsi inti `count_hits(n, tid)` yang membuat RNG **lokal** pada *stack* thread itu, dengan seed turunan dari (42, tid). Tidak ada satu pun state RNG yang dibagi antar thread.

```
typedef struct {            // satu cache line (64 B) per thread
    int tid; uint64_t n; char pad0[48];
    uint64_t hits;          // hasil lokal thread
    char pad1[56];
} task_t;

static void *worker(void *arg) {
    task_t *t = (task_t *)arg;
    t->hits = count_hits(t->n, (uint64_t)t->tid);   // RNG lokal
    return NULL;
}
```

Mekanisme sinkronisasi

Pendekatan yang dipilih adalah **reduction manual tanpa kunci**. Setiap thread menulis hasilnya hanya ke slot `tasks[tid].hits` miliknya sendiri; tidak ada dua thread yang menulis lokasi memori yang sama sehingga *race condition* pada akumulator tidak mungkin terjadi. Thread utama memanggil `pthread_join()` untuk setiap thread

(berfungsi sekaligus sebagai *barrier* dan menjamin *happens-before* antara penulisan hasil worker dan pembacaan oleh thread utama), kemudian menjumlahkan semua slot secara serial: sebuah penjumlahan T bilangan, yang biayanya dapat diabaikan.

Alternatif yang ditolak: (a) `pthread_mutex_lock` di dalam loop, yang akan men-serialisasi 10^9 operasi dan menghancurkan *speedup*; (b) variabel global bersama dengan `__atomic_fetch_add` per iterasi, yang menyebabkan *cache-line bouncing* antar core; (c) satu kali *lock* di akhir tiap thread, yang benar namun tidak perlu karena `join` sudah cukup.

False sharing. Bila slot hasil disimpan dalam array `uint64_t hits[T]` yang rapat, beberapa slot berbagi satu *cache line* 64 byte. Penulisan oleh satu core meng-invalidate baris itu pada core lain walaupun variabelnya berbeda. Karena ditulis sekali di akhir, dampaknya pada program ini kecil, tetapi *padding* diterapkan sebagai praktik yang benar dan dapat dijelaskan secara eksplisit. Pada versi yang menaikkan `hits` langsung di slot array di dalam loop, *false sharing* dapat menurunkan kinerja berkali lipat.

2.3 Strategi Paralelisasi OpenMP

Pemilihan pragma & clause

```
uint64_t hits = 0;
#pragma omp parallel num_threads(T) default(none) shared(N, T) reduction(+:hits)
{
    int      tid = omp_get_thread_num();       // private (dideklarasi di dalam region)
    uint64_t n   = chunk_size(N, T, tid);      // private
    hits += count_hits(n, (uint64_t)tid);      // RNG lokal di dalam count_hits
}
```

Cukup satu direktif. Alasan pemilihan tiap bagian:

Direktif / clause	**Fungsi**	**Alasan dipilih**
parallel	Membentuk tim T thread yang menjalankan blok yang sama (model SPMD).	Memungkinkan setiap thread membuat RNG sendiri dan memakai partisi eksplisit identik dengan Pthreads.
num_threads(T)	Menetapkan jumlah thread.	Variasi 1, 2, 4, 8 dikendalikan dari argumen program, bukan variabel lingkungan.
default(none)	Mewajibkan semua variabel dideklarasikan scope-nya.	Mencegah scope tersirat yang keliru (sumber bug umum); kompilator memaksa kita memikirkan setiap variabel.
shared(N, T)	N dan T dibagi antar thread.	Hanya dibaca, tidak ada penulisan sehingga aman tanpa sinkronisasi.
reduction(+:hits)	Tiap thread memperoleh salinan privat *hits* (inisial 0); saat region berakhir semua salinan dijumlahkan ke variabel asli.	Menghilangkan race condition pada akumulator secara otomatis dan tanpa kunci di dalam loop; setara dengan reduction manual pada Pthreads.

Mengapa bukan `#pragma omp parallel for`**?** Bentuk *parallel for* dengan satu RNG per thread dan `schedule(static)` secara fungsional juga benar, tetapi pembagian iterasi dan seed-nya ditentukan runtime sehingga hasil numerik tidak otomatis identik dengan versi Pthreads. Dengan membentuk partisi dan seed secara eksplisit di dalam `parallel`, kedua implementasi menghasilkan *hits* yang **persis sama** untuk T yang sama, sehingga verifikasi correctness menjadi tepat (bit-per-bit), bukan sekadar statistik, dan perbandingan kinerja menjadi adil (kerja identik).

Scope variabel (shared / private)

Variabel	Scope	Keterangan
N, T	shared	Konstanta masukan; hanya dibaca.
tid, n	private	Dideklarasikan di dalam blok parallel sehingga otomatis privat per thread.
rng (di dalam count_hits)	private	Variabel lokal fungsi, berada pada stack tiap thread; tidak pernah dibagi.
hits	reduction (private selama region, digabung di akhir)	Salinan privat per thread, dijumlahkan oleh runtime saat *implicit barrier* di akhir region.

2.4 Kebijakan RNG per Thread (Berlaku untuk Kedua Versi)

Seed thread ke-t dihitung sebagai `seed = 42 XOR ((t+1) × 0xD1B54A32D192ED03)`, kemudian diperluas menjadi 4 × 64 bit state dengan SplitMix64. Konstanta pengali yang besar dan ganjil memastikan seed antar thread berbeda jauh dalam representasi bit, dan SplitMix64 (fungsi *mixing* yang kuat) menghilangkan korelasi di antara state awal. Dengan demikian aliran acak antar thread praktis independen. Perlu dicatat bahwa pendekatan seed-berbeda tidak menjamin secara matematis bahwa aliran tidak bertumpang tindih; pendekatan yang lebih ketat adalah memakai fungsi *jump()* pada xoshiro yang melompat 2^{128} langkah. Untuk 10^9 sampel dalam ruang periode 2^{256} kemungkinan tumpang tindih dapat diabaikan, dan trade-off ini dicatat sebagai keterbatasan.

3. HASIL PENGUJIAN

3.1 Lingkungan Pengujian

Komponen	Spesifikasi
Prosesor	AMD Ryzen 5 7535HS with Radeon Graphics, 3,30 GHz (hingga 4,55 GHz)
Core fisik / thread logis	6 core fisik / 12 thread logis (SMT)
RAM	16 GB DDR5
Sistem operasi	Windows 11 Home (64-bit, Build 26200)
Kompilator	gcc (MinGW-w64), flag `-O2 -fopenmp -pthread`; [ISI: versi gcc dari **gcc --version**]
Parameter	N = 10^9 titik, seed dasar 42, T ∈ {1, 2, 4, 8}, tiap konfigurasi diulang 3 kali dan dirata-rata

Waktu diukur dengan timer monotonic (`clock_gettime(CLOCK_MONOTONIC)` pada Linux, `QueryPerformanceCounter` pada Windows) hanya di sekitar bagian komputasi (pembuatan thread hingga reduction selesai); pencetakan hasil tidak ikut diukur.

3.2 Verifikasi Correctness

Verifikasi dilakukan dengan skrip `verify.sh` (N = 10^7) yang membandingkan jumlah *hits* antar versi. Kriteria: (i) serial = Pthreads (T=1) = OpenMP (T=1); (ii) Pthreads = OpenMP untuk setiap T. Karena kerja dan seed identik, kesamaan harus persis, bukan hanya mendekati.

Versi	Hits (N = 10^7)	Status
Serial	7.854.510	Baseline
Pthreads / OpenMP, T=1	7.854.510 / 7.854.510	Sama dengan serial
Pthreads / OpenMP, T=2	7.854.656 / 7.854.656	Pthreads = OpenMP
Pthreads / OpenMP, T=4	7.852.181 / 7.852.181	Pthreads = OpenMP
Pthreads / OpenMP, T=8	7.852.630 / 7.852.630	Pthreads = OpenMP

[ISI: tempel keluaran verifikasi dari mesin Anda, yaitu baris T=... -> OK pada bench.ps1]

Hasil *hits* untuk T > 1 berbeda dari serial. Ini **bukan kesalahan**: dengan T thread, sampel berasal dari T aliran RNG berbeda, bukan satu aliran panjang. Hasilnya tetap deterministik dan dapat direproduksi untuk T tertentu, dan kebenarannya dinilai secara statistik (Subbab 3.5), bukan identitas bit terhadap serial. Estimasi π untuk semua baris di atas berkisar 3,1409–3,1419, konsisten dengan galat standar ≈ $5{,}2 \times 10^{-4}$ pada N = 10^7. Perlu diakui bahwa uji ini tidak dapat mendeteksi *race condition* yang bermanifestasi sebagai hasil acak berbeda tiap run; karena tidak ada variabel yang ditulis bersama, desainnya aman secara konstruksi, dan hasil identik antar run berulang menjadi bukti pendukung.

3.3 Tabel Pengukuran Waktu Eksekusi

Implementasi	Thread	Waktu rata-rata (s)	Std (s)	Speedup	Efficiency
Serial	1	[ISI: detik]	-	1,00	1,00
Pthreads	1	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
OpenMP	1	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]

Implementasi	Thread	Waktu rata-rata (s)	Std (s)	Speedup	Efficiency
Pthreads	2	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
OpenMP	2	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
Pthreads	4	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
OpenMP	4	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
Pthreads	8	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]
OpenMP	8	[ISI: detik]	[ISI: std]	[ISI: S]	[ISI: E]

Tabel 1. Waktu eksekusi $N = 10^9$ (rata-rata 3 kali run). Speedup $S(T) = T_{serial} / T_{paralel}(T)$; Efficiency $E(T) = S(T) / T$.

3.4 Grafik Speedup & Efficiency

[ISI: sisipkan speedup_efficiency.png hasil **python3 plot.py** setelah **make bench**]

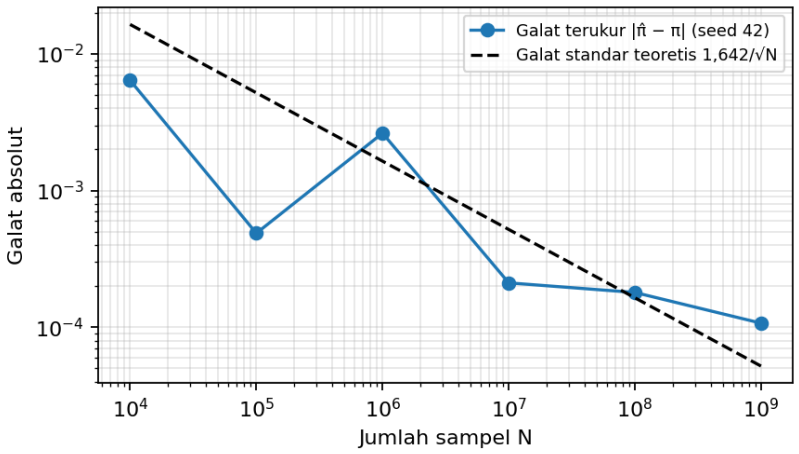
Gambar 1. Speedup dan Efficiency terhadap jumlah thread.

3.5 Pengujian Variasi Thread dan Akurasi Statistik

Selain waktu, dikaji hubungan antara jumlah sampel/thread dan akurasi. Eksperimen berikut dijalankan dengan seed tetap (42) dan ditampilkan apa adanya.

N	Hits	π̂	Galat $\lvert\hat{\pi} - \pi\rvert$	Galat standar teoretis 1,642/√N
10.000	7.870	3,148000	6,41e-03	1,64e-02
100.000	78.552	3,142080	4,87e-04	5,19e-03
1.000.000	784.741	3,138964	2,63e-03	1,64e-03
10.000.000	7.854.510	3,141804	2,11e-04	5,19e-04
100.000.000	78.535.323	3,141413	1,80e-04	1,64e-04
1.000.000.000	785.371.388	3,141486	1,07e-04	5,19e-05

Tabel 2. Akurasi terhadap N (serial).



Gambar 2. Galat terukur terhadap galat standar teoretis (skala log-log).

Thread	Hits (N = 10^8)	π̂	Galat $\lvert\hat{\pi} - \pi\rvert$
1	78.535.323	3,141413	1,80e-04
2	78.539.302	3,141572	2,06e-05

Thread	**Hits (N = 10^8)**	**$\hat{\pi}$**	**Galat $\lvert\hat{\pi} - \pi\rvert$**
4	78.545.482	3,141819	2,27e-04
8	78.541.311	3,141652	5,98e-05

Tabel 3. Akurasi terhadap jumlah thread pada total sampel tetap $N = 10^8$ (Pthreads).

Pengamatan. Galat terukur mengikuti tren penurunan $1/\sqrt{N}$ dan berada dalam kisaran beberapa kali galat standar teoretis pada semua N. Pada $N = 10^9$ galat terukur $1{,}07 \times 10^{-4}$ sekitar 2,1 kali galat standar ($5{,}2 \times 10^{-5}$); ini masih wajar secara statistik (kejadian dua sigma terjadi sekitar 4% dari run), tetapi karena hanya satu seed yang diuji, kesimpulan yang lebih kuat memerlukan pengulangan dengan beberapa seed. Pada Tabel 3, galat untuk T = 1, 2, 4, 8 bervariasi acak ($2{,}1 \times 10^{-5}$ hingga $2{,}3 \times 10^{-4}$) tanpa tren monoton terhadap T. Ini sesuai teori: pada total sampel tetap, jumlah thread tidak mengubah galat standar; thread hanya mengubah *waktu*. Variasi antar baris adalah fluktuasi sampling dari aliran RNG yang berbeda.

4. ANALISIS KRITIS

4.1 Perbandingan Kompleksitas Kode (LOC & Maintenance)

LOC dihitung dari baris kode efektif (tanpa baris kosong dan komentar) pada file utama tiap versi. File `common.h` (70 baris, termasuk blok kompatibilitas Windows) dan `report.h` (30 baris) dipakai bersama oleh semua versi sehingga tidak dihitung sebagai biaya paralelisasi.

Versi	LOC efektif (file utama)	Tambahan vs serial	API / konsep yang harus dikelola
Serial	11	-	-
Pthreads	38	+27 (≈ 3,5 × serial)	pthread_t, pthread_create/join, struct argumen, padding cache line, alokasi array, penanganan error, reduction manual
OpenMP	18	+7 (≈ 1,6 × serial)	satu pragma dengan clause; omp_get_thread_num

Tabel 4. Perbandingan LOC. Pthreads membutuhkan sekitar 3,9 kali tambahan kode dibanding OpenMP untuk fungsionalitas yang sama.

Selisih ini kecil dalam angka absolut karena masalahnya sederhana, tetapi kualitatifnya penting. Pada Pthreads, programmer bertanggung jawab atas: membungkus argumen ke dalam struct dan `void*` (hilang *type safety*), siklus hidup thread (create/join dan kebocoran bila lupa join), pengecekan kegagalan `pthread_create`, alokasi dan pembebasan memori, serta reduction. Setiap bagian adalah titik potensi bug. Pada OpenMP, program serial tetap hampir utuh: bila direktif dihapus atau kompilator tidak mendukung OpenMP, program berjalan serial dengan hasil yang sama sehingga **jalur serial menjadi referensi uji bawaan**. Ini keunggulan *maintainability* yang nyata: pemeliharaan cukup membaca satu pragma, sedangkan versi Pthreads harus dibaca sebagai kode konkurensi penuh. Sebaliknya, kesederhanaan sintaks OpenMP menyembunyikan perilaku runtime (bagaimana reduction digabung, kapan thread dibuat, pengikatan thread ke core), sehingga bug scope atau sinkronisasi yang halus (mis. lupa `default(none)` dan variabel menjadi shared secara tersirat) dapat tidak terlihat.

4.2 Analisis Bottleneck (Hukum Amdahl)

Hukum Amdahl menyatakan bahwa dengan fraksi paralel p, speedup maksimum pada n prosesor adalah $S(n) = 1 / ((1 - p) + p/n)$. Pada program ini, bagian serial hanya: pembuatan/penggabungan thread, inisialisasi RNG, reduction T bilangan, dan pembacaan argumen. Biaya semuanya pada orde mikrodetik hingga milidetik, sedangkan loop utama 10^9 iterasi berlangsung beberapa detik, sehingga p sangat dekat 1.

p (fraksi paralel)	S(2)	S(4)	S(8)	S(∞)
0.9999	2,00	4,00	7,99	10.000
0.999	2,00	3,99	7,94	1.000
0.99	1,98	3,88	7,48	100

Tabel 5. Speedup teoretis Amdahl. Bahkan p = 0,99 (1% serial) hanya memberi S(8) ≈ 7,5, sehingga bila speedup terukur jauh di bawah ini, penyebabnya bukan fraksi serial program.

Bila speedup terukur lebih rendah daripada ideal, **Hukum Amdahl bukan penjelasan utama** karena fraksi serial program ini hampir nol. Faktor pembatas yang lebih mungkin pada beban *compute-bound* seperti ini adalah:

- **Jumlah core fisik vs thread logis.** Pada CPU dengan SMT/Hyper-Threading (mis. 4 core / 8 thread), dua thread berbagi satu core dan unit eksekusi yang sama. Loop xoshiro yang padat ALU hanya memperoleh tambahan throughput sekitar 10–30% dari thread SMT kedua, bukan 100%. Karena itu T = 8 pada mesin 4-core tidak akan mencapai speedup 8.
- **Frekuensi turbo yang turun.** Satu core aktif bisa berjalan pada frekuensi turbo tertinggi; semua core aktif memaksa frekuensi lebih rendah (dan *thermal throttling* pada laptop). Waktu serial pembanding menjadi 'lebih cepat per-core' daripada kondisi paralel penuh.
- **Gangguan sistem operasi dan proses latar.** Penjadwal dapat menempatkan dua thread pada core yang sama atau berbagi core dengan proses lain, terutama saat T sama dengan jumlah core logis.
- **Memory bandwidth bukan bottleneck.** Seluruh state berada di register dan L1, tidak ada lalu lintas memori yang berarti; ini diperkirakan membuat *scaling* lebih baik daripada program berbasis data besar seperti perkalian matriks.

Untuk mengukur fraksi serial *efektif* dari data nyata digunakan metrik Karp-Flatt: e = (1/S − 1/n) / (1 − 1/n). Nilai e yang kecil dan konstan terhadap n menandakan overhead kecil; e yang naik bersama n menandakan overhead yang tumbuh.

Implementasi	T	Speedup	Efficiency	Karp-Flatt e
Pthreads	2	[ISI: S]	[ISI: E]	[ISI: e]
Pthreads	4	[ISI: S]	[ISI: E]	[ISI: e]
Pthreads	8	[ISI: S]	[ISI: E]	[ISI: e]
OpenMP	2	[ISI: S]	[ISI: E]	[ISI: e]
OpenMP	4	[ISI: S]	[ISI: E]	[ISI: e]
OpenMP	8	[ISI: S]	[ISI: E]	[ISI: e]

Tabel 6. Metrik Karp-Flatt (hitung dari Tabel 1).

Perlu juga dicatat **Hukum Gustafson**: karena N dapat diperbesar mengikuti jumlah core (*scaled speedup*), masalah Monte Carlo hampir selalu *scalable*; yang dibatasi bukan paralelisme, tetapi perangkat keras. Tugas ini mengukur *strong scaling* (N tetap), yang lebih ketat dibanding weak scaling.

4.3 Pembahasan Anomali

Bagian ini mencatat penyimpangan dari perilaku ideal pada data pengukuran, penyebab yang paling mungkin, dan cara memeriksanya.

[ISI: tuliskan anomali NYATA dari Tabel 1 setelah benchmark selesai, mis. speedup T=8 jauh dari ideal dan hubungannya dengan jumlah core fisik, atau selisih Pthreads vs OpenMP]

Daftar pemeriksaan umum untuk anomali sejenis:

Kemungkinan anomali	**Penyebab**	**Cara memverifikasi**
Speedup T=8 jauh di bawah 8	Core fisik < 8 (SMT), turbo turun, throttling.	`lscpu` (Core(s) per socket, Thread(s) per core); bandingkan T=4 dan T=8; pantau suhu/frekuensi.
Pthreads T=1 sedikit lebih lambat dari serial	Overhead create/join thread dan penyebaran data; juga optimasi kompilator berbeda pada loop dalam fungsi worker.	Bandingkan serial vs T=1; bandingkan hasil `-O2` vs `-O3`.
OpenMP T=1 berbeda dari serial	Inisialisasi runtime libgomp (pembuatan thread pool) dihitung pada run pertama.	Jalankan satu run *warm-up* yang dibuang.
Pthreads vs OpenMP selisih kecil (< 2–3%)	Keduanya menjalankan loop yang sama; selisih mayoritas berasal dari derau pengukuran.	Periksa simpangan baku; bila selisih < std maka tidak signifikan.
Std besar / satu run outlier	Proses latar, penjadwal, throttling.	Naikkan jumlah ulangan; laporkan median selain rata-rata.
Speedup > T (superlinear)	Hampir tidak mungkin pada beban ini (tidak ada efek cache); biasanya artefak turbo/pengukuran serial yang lambat.	Ukur ulang serial setelah mesin dingin.

Satu anomali lain dari eksperimen akurasi (Subbab 3.5): galat pada N = 10^9 sekitar 2,1 kali galat standar teoretis. Secara statistik masih wajar (peluang ≈ 4% untuk $|z| \geq 2{,}1$), tetapi hanya satu seed yang diuji sehingga dianjurkan mengulang dengan beberapa seed.

4.4 *Trade-off* Jumlah Thread vs Akurasi Statistik

Galat standar estimator hanya bergantung pada N total: $SE = 4\sqrt{(p(1-p)/N)} \approx 1{,}642/\sqrt{N}$ dengan $p = \pi/4$. Jumlah thread **tidak** memperbaiki akurasi; ia hanya memperpendek waktu untuk mencapai akurasi tersebut. Dengan anggaran waktu tetap, T thread memungkinkan sekitar T × lebih banyak sampel sehingga galat turun sekitar $1/\sqrt{T}$ (Hukum Gustafson). Misalnya 8 thread dengan waktu sama menghasilkan galat ≈ 2,8 kali lebih kecil, bukan 8 kali. Trade-off lain muncul pada kualitas RNG: bila aliran antar thread berkorelasi (mis. seed berurutan pada generator lemah, atau RNG global yang dilock sehingga ada urutan non-deterministik), sampel efektif lebih sedikit daripada N dan galat nyata lebih besar dari $1/\sqrt{N}$. Itu sebabnya RNG privat dengan seed terpisah dan pencampuran SplitMix64 dipakai di sini. Tabel 3 mendukung hal ini: galat pada N tetap bervariasi acak terhadap T tanpa tren monoton.

4.5 *Trade-off* Kemudahan vs Kontrol

Aspek	**Pthreads**	**OpenMP**
Kemudahan menulis	Rendah: semua manajemen thread manual	Tinggi: 1 pragma, jalur serial tetap utuh
Kontrol partisi & scheduling	Penuh: bentuk apa pun (pool, work-stealing)	Terbatas pada schedule(static/dynamic/guided), task
Sinkronisasi	Mutex, condvar, barrier, atomics, semaphore, bebas memilih	critical, atomic, barrier, reduction; lebih ringkas, kurang fleksibel

Aspek	**Pthreads**	**OpenMP**
Risiko bug	Tinggi (leak, deadlock, lupa join, salah pointer argumen)	Lebih rendah, tetapi scope tersirat dan data race tetap mungkin
Portabilitas	POSIX (Linux/macOS; Windows lewat pustaka tambahan)	Lintas platform lewat kompilator yang mendukung OpenMP
Kinerja (beban reguler)	Setara bila dirancang benar	Setara; runtime telah dioptimasi (thread pool)
Cocok untuk	Struktur konkurensi tak beraturan, server, pipeline	Paralelisme data/loop pada komputasi numerik

Tabel 7. Ringkasan trade-off.

Untuk Monte Carlo, kontrol ekstra Pthreads tidak memberi keuntungan kinerja karena beban seragam dan pembagian statis sudah optimal. Satu-satunya keunggulan Pthreads di sini adalah transparansi: setiap detail (padding, join sebagai barrier, reduction) terlihat dan dapat dijelaskan, sehingga baik secara edukatif. Untuk rekayasa nyata, OpenMP memberikan hasil sama dengan usaha jauh lebih kecil.

5. KESIMPULAN

5.1 Rekomendasi Penggunaan Pthreads vs OpenMP

- **Gunakan OpenMP** untuk paralelisme data berbasis loop pada komputasi numerik seperti Monte Carlo, perkalian matriks, dan konvolusi: kodenya lebih pendek (18 vs 38 LOC pada program ini), jalur serial tetap tersedia sebagai referensi uji, dan risiko bug kinerja lebih rendah. dan kinerja setara pada beban reguler.
- **Gunakan Pthreads** bila memerlukan kontrol penuh atas siklus hidup thread, pola sinkronisasi khusus (producer-consumer, pool, pipeline), prioritas atau afinitas, atau bila lingkungan tidak menyediakan kompilator OpenMP.
- Apa pun pendekatannya: gunakan RNG privat per thread dengan generator berkualitas, hindari kunci pada jalur panas, gunakan reduction, dan waspadai false sharing.

5.2 Pembelajaran yang Didapat

- Race condition pada akumulator dihilangkan dengan struktur data (hasil per thread + join/reduction), bukan dengan kunci di loop.
- Menyamakan partisi dan seed antar implementasi membuat verifikasi correctness bersifat tepat (bit-per-bit), sehingga perbandingan kinerja menjadi adil.
- Akurasi Monte Carlo ditentukan oleh N total (≈ 1/√N), bukan jumlah thread; paralelisasi memberi lebih banyak sampel per satuan waktu.
- Speedup nyata dibatasi perangkat keras (core fisik, SMT, turbo), bukan oleh fraksi serial program, karena p mendekati 1; metrik Karp-Flatt berguna untuk mendiagnosis hal ini.
- Kualitas RNG adalah bagian dari correctness program paralel: RNG global berkunci lambat dan mengganggu independensi sampel.

Keterbatasan: pengujian memakai satu seed dasar dan satu mesin; pembangkitan seed per thread tidak menjamin ketiadaan tumpang tindih aliran secara matematis (jump function xoshiro dapat dipakai); dan pengukuran bergantung pada kondisi termal serta beban sistem saat pengujian.

6. DAFTAR PUSTAKA

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*, 30, 483–485.

[2] Blackman, D., & Vigna, S. (2021). Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software*, 47(4), 1–32.

[3] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.

[4] Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[5] Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532–533.

[6] Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539–543.

[7] Metropolis, N., & Ulam, S. (1949). The Monte Carlo method. *Journal of the American Statistical Association*, 44(247), 335–341.

[8] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org

[9] Pacheco, P. S., & Malensek, M. (2021). *An Introduction to Parallel Programming* (2nd ed.). Morgan Kaufmann.

[10] Steele, G. L., Lea, D., & Flood, C. H. (2014). Fast splittable pseudorandom number generators. *Proceedings of OOPSLA 2014*, 453–472.

[11] The Open Group. (2018). *POSIX.1-2017 (IEEE Std 1003.1-2017): pthread.h*. https://pubs.opengroup.org/onlinepubs/9699919799/